

Collegium Witelona

Projektowanie i programowanie systemów internetowych I

U Alchemika System

Aplikacja internetowa obiektu agroturystycznego

Dokumentacja projektowa

Autorzy: Maciej Bułhak

Małgorzata Bułhak

Hubert Olszak

Przedmiot: Projektowanie i programowanie
systemów internetowych I

Rok akademicki: 2025/2026

Data: 10 czerwca 2026

Stos technologiczny: Django 4.2 (MVT) + PostgreSQL 16
+ Bootstrap 5 + HTMX + Docker

Repozytorium: <https://github.com/Bulileusz/u-alchemika-system>

Spis treści

1	Wprowadzenie	2
1.1	Cel dokumentu	2
1.2	Kontekst projektu	2
1.3	Zespół projektowy	2
2	Charakterystyka systemu	2
2.1	Cel i zakres	2
2.2	Aktorzy	3
2.3	Mapa funkcji	3
2.4	Kluczowy przepływ: wysłanie zapytania	3
3	Architektura aplikacji	4
3.1	Wzorzec MVT	4
3.2	Podział na aplikacje	4
3.3	Struktura repozytorium	4
4	Model danych (skrót)	5
5	Warstwa prezentacji	6
5.1	Szablony i dziedziczenie	6
5.2	Style i responsywność (RWD)	6
5.3	Interaktywność po stronie klienta	6
5.4	Asynchroniczność (HTMX)	6
6	Logika aplikacji	7
6.1	Formularze i walidacja	7
6.2	Dostęp do danych i optymalizacja ORM	7
6.3	Cache	8
6.4	Poczta	8
6.5	Uwierzytelnianie i panel obsługi	9
6.6	Rejestrowanie zdarzeń (logger)	9
7	Internacjonalizacja (PL/DE)	9
8	Konfiguracja i bezpieczeństwo	10
9	Konteneryzacja i uruchomienie	10
9.1	Środowisko (Docker Compose)	10
9.2	Uruchomienie lokalne	10
9.3	Wdrożenie zdalne	11
10	Podsumowanie i decyzje projektowe	12
A	Skorowidz zagadnień warstwy internetowej	13

1 Wprowadzenie

1.1 Cel dokumentu

Niniejszy dokument jest **dokumentacją projektową** aplikacji internetowej „U Alchemika” – witryny obiektu agroturystycznego położonego w Górach Izerskich. Opisuje on system od strony funkcjonalnej i technicznej: co aplikacja robi, jak jest zbudowana, z jakich technologii korzysta oraz jak uruchomić ją lokalnie i wdrożyć na serwerze. Dokument adresowany jest do osoby, która ma zrozumieć działanie systemu, rozwijać go lub uruchomić od zera.

1.2 Kontekst projektu

Aplikacja powstała w ramach kursu *Projektowanie i programowanie systemów internetowych I* i współdzieli repozytorium oraz kod z kursem *Projektowanie systemów baz danych*. Oba przedmioty opisują ten sam system z dwóch perspektyw. Warstwa danych – model relacyjny, klucze obce, indeksy, historia migracji – została szczegółowo udokumentowana w osobnym opracowaniu (`docs/dokumentacja.tex`). **Niniejszy dokument koncentruje się na warstwie internetowej**: funkcjach widocznych dla użytkownika, architekturze aplikacji webowej oraz technologiach frontendu i backendu. Model danych omówiono tu skrótowo (sekcja 4), bez powielania dokumentacji bazodanowej.

1.3 Zespół projektowy

Osoba	Zakres odpowiedzialności
Maciej Bułhak	Model danych, migracje i integralność, warstwa dostępu do danych (ORM), konfiguracja projektu, konteneryzacja, dokumentacja.
Małgorzata Bułhak	Moduł zapytań rezerwacyjnych (formularz, walidacja, poczta), panel obsługi, logowanie zdarzeń, dane obiektu i kontakt.
Hubert Olszak	Warstwa prezentacji: szablony HTML, style CSS, responsywność, interaktywność po stronie klienta (JavaScript), blog i atrakcje, dane testowe.

2 Charakterystyka systemu

2.1 Cel i zakres

U Alchemika to witryna prezentująca ofertę noclegową obiektu i zbierająca **zapytania rezerwacyjne** od gości. Przyjęto świadome założenie projektowe, że system **nie realizuje pełnej rezerwacji online** – gość wysyła zapytanie, a finalizacja odbywa się poza systemem (kontakt bezpośredni lub przekierowanie do serwisu Booking.com). Dzięki temu zakres jest spójny i kompletny w obrębie zdefiniowanych przypadków użycia, zamiast obejmować pobieżnie zaimplementowany moduł płatności i kalendarza dostępności.

2.2 Aktorzy

Gość (użytkownik anonimowy) – przegląda ofertę (pokoje, galeria, atrakcje, blog), zapoznaje się z polityką obiektu i wysyła zapytanie rezerwacyjne. Nie wymaga konta ani logowania.

Obsługa / administrator – po uwierzytelnieniu przegląda nadesłane zapytania w panelu wewnętrznym, a całością danych (pokoje, treści, zapytania) zarządza przez panel Django Admin.

2.3 Mapa funkcji

Tabela 2 zestawia wszystkie trasy aplikacji wraz z ich funkcją i wymaganym poziomem dostępu.

Tabela 2: Mapa stron aplikacji

Strona	Adres (URL)	Funkcja	Dostęp
Strona główna	/	Hero, polecane pokoje, zjawka atrakcji, CTA kontaktu.	publiczny
Lista pokoi	/pokoje/	Karty aktywnych pokoi z ceną i pojemnością.	publiczny
Szczegóły pokoju	/pokoje/<slug>/	Galeria, opis, udogodnienia, cena, przejście do zapytania.	publiczny
Galeria	/galeria/	Zdjęcia obiektu z powiększaniem (lightbox).	publiczny
Atrakcje	/atrakcje/	Atrakcje turystyczne okolicy.	publiczny
Blog	/blog/	Lista opublikowanych wpisów.	publiczny
Wpis blogowy	/blog/<slug>/	Treść pojedynczego wpisu.	publiczny
Kontakt	/kontakt/	Dane kontaktowe obiektu (z cache).	publiczny
Obiekt	/obiekt/	Polityki obiektu (doba, zwierzęta, płatności).	publiczny
Zapytanie	/zapytanie/	Formularz zapytania rezerwacyjnego (HTMX).	publiczny
Logowanie	/login/	Uwierzytelnienie obsługi.	publiczny
Panel zapytań	/panel/zapytania/	Lista nadesłanych zapytań.	chroniony
Panel admina	/admin/	Pełny CRUD na wszystkich encjach.	chroniony
Zmiana języka	/i18n/setlang/	Przełączenie interfejsu PL↔DE.	publiczny

2.4 Kluczowy przepływ: wysłanie zapytania

Najważniejszy scenariusz biznesowy łączy kilka mechanizmów technicznych w jednej interakcji użytkownika:

1. Gość wypełnia formularz na stronie `/zapytanie/` (opcjonalnie wywołany z karty konkretnego pokoju, który zostaje podstawiony jako wartość początkowa pola).
2. Formularz jest wysyłany **asynchronicznie** (HTMX) – bez przeładowania strony.

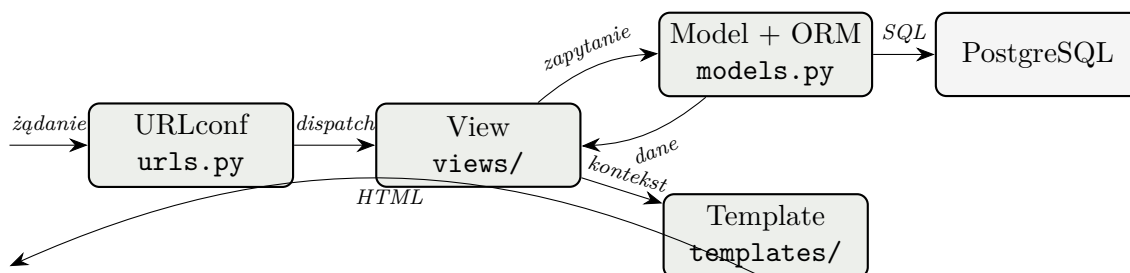
3. Serwer waliduje dane (poprawność e-maila, kolejność i długość dat pobytu).
4. Po powodzeniu zapytanie zostaje **zapisane w bazie**, na adres gościa trafia **e-mail potwierdzający**, a zdarzenie zostaje **zalogowane**.
5. Gość widzi sekcję podziękowania – fragment HTML podmieniony w miejscu formularza przez HTMX.
6. Zapytanie pojawia się na liście w panelu obsługi `/panel/zapytania/`.

Ten pojedynczy przepływ angażuje jednocześnie: formularz z walidacją, komunikację asynchroniczną, zapis do bazy przez ORM, wysyłkę poczty oraz rejestrację zdarzeń w logach – dlatego stanowi dobry punkt odniesienia dla pozostałych rozdziałów.

3 Architektura aplikacji

3.1 Wzorzec MVT

Aplikację zbudowano na frameworku **Django 4.2**, który realizuje wzorzec **MVT** (Model–View–Template) – odmianę MVC, w której rolę kontrolera pełni *View*, a prezentację opisuje *Template*. Przepływ pojedynczego żądania HTTP przedstawia rysunek 1.



Rysunek 1: Przepływ żądania HTTP w architekturze MVT

- **Model** – definicje encji i reguł biznesowych mapowane przez ORM na tabele (`apps/core/models.py`, `apps/content/models.py`).
- **View** – logika obsługi żądania: pobranie danych, walidacja, wybór szablonu (`apps/core/views/`, `apps/content/views.py`).
- **Template** – szablony HTML z dziedziczeniem i partialami (`backend/templates/` oraz katalogi `templates/` aplikacji).

3.2 Podział na aplikacje

Kod podzielono na dwie aplikacje Django o rozłącznej odpowiedzialności, co ograniczyło konflikty przy pracy zespołowej (każda osoba pracowała głównie we własnym obszarze):

- `apps/core` – rdzeń systemu: pokoje, udogodnienia, galeria, zapytania rezerwacyjne, dane obiektu i kontakt, logi audytu,
- `apps/content` – treści marketingowe: wpisy blogowe i atrakcje okolicy (aplikacja niezależna, bez powiązań z `core`).

3.3 Struktura repozytorium

```

u-alchemika-system/
|- backend/
|  |- config/                # settings.py, urls.py, wsgi.py
|  |- apps/
|     |- core/               # pokoje, zapytania, kontakt, galeria
|     |  |- views/          # pakiet: rooms.py, inquiries.py
|     |  |- models.py forms.py urls.py admin.py
|     |  '- static/ templates/ # zasoby i szablony aplikacji
|     '- content/           # blog + atrakcje (osobna apka)
|- templates/               # base.html, index.html (wspolne)
|- static/                  # style.css, obrazy
|- locale/de/               # katalog tlumaczen niemieckich
|- requirements.txt          # zaleznosci Pythona
|- '- Dockerfile
|- docs/                    # dokumentacja (ten plik + dok. bazy
    danych)
'- docker-compose.yml       # web + db + mailhog

```

Listing 1: Najważniejsze elementy drzewa katalogów

Routing jest hierarchiczny: główny `config/urls.py` deleguje przez `include()` do plików `urls.py` obu aplikacji oraz montuje widoki logowania i przełącznika języka.

4 Model danych (skrót)

Dane przechowuje **PostgreSQL 16**; dostęp realizuje wyłącznie **Django ORM** – w kodzie nie ma ręcznie pisanego SQL. Tabela 3 wymienia encje systemu. Pełny model relacyjny (klucze obce, reguły kasowania, indeksy, historia migracji) opisano w dokumentacji bazodanowej.

Tabela 3: Encje systemu

Encja	Rola
Room	Pokój: nazwa, slug, opis, pojemność, cena, status aktywności.
RoomImage	Zdjęcie pokoju (relacja 1:N, kolejność wyświetlania).
Amenity	Udogodnienie (np. WiFi, kominek).
RoomAmenity	Tabela pośrednia relacji M:N pokój–udogodnienie.
Inquiry	Zapytanie rezerwacyjne gościa (FK do pokoju, status, daty).
PropertyInfo	Dane i polityki obiektu (kontakt, doba hotelowa, płatności).
AuditLog	Rejestr zdarzeń (FK do użytkownika).
Post	Wpis blogowy (publikowany / szkic).
Attraction	Atrakcja turystyczna okolicy.

Optymalizację zapytań pod kątem problemu N+1 omówiono w sekcji 6.2.

5 Warstwa prezentacji

5.1 Szablony i dziedziczenie

Widok HTML budują szablony Django. Wspólny `base.html` zawiera nawigację, stopkę, dołączenie Bootstrapa i HTMX oraz blok na komunikaty; podstrony rozszerzają go (`{% extends %}`) i nadpisują blok treści. Powtarzalne fragmenty wydzielono do **partiali** dołączanych przez `{% include %}` – dzięki temu strona główna powstaje ze złożenia niezależnych części, a każdy autor dostarczał własny partial bez kolizji:

```
{% include 'core/_featured_rooms.html' %}      {# polecane pokoje
#}
{% include 'content/_attractions_teaser.html' %} {# zajawka
atrakcji #}
{% include 'core/_contact_cta.html' %}          {# wezwanie do
kontaktu #}
```

Listing 2: Składanie strony głównej z partiali (`index.html`)

5.2 Style i responsywność (RWD)

Warstwę CSS oparto na **Bootstrap 5.3** (siatka, karty, przyciski, alerty, klasy walidacji formularza) dołączonym z CDN, uzupełnionym o własny arkusz `static/css/style.css` nadający charakter marce (navbar z logo, sekcja hero, karty pokoi, galeria). Responsywność zapewniają: meta `viewport`, responsywna siatka Bootstrap (karty pokoi układają się 3 w rzędzie na desktopie i 1 na telefonie) oraz własne media queries. Pasek nawigacji wraz z przełącznikiem języka zawija się na węższych ekranach.

5.3 Interaktywność po stronie klienta

Skrypt `apps/core/static/core/js/inquiry.js` wzbogaca formularz zapytania: licznik znaków wiadomości, walidacja dat po stronie klienta (minimalna data = dziś, data wyjazdu po dacie przyjazdu) oraz ponowne podpięcie tych zdarzeń po asynchronicznej podmianie formularza przez HTMX (nasłuch zdarzenia `htmx:afterSwap`). Galeria zdjęć (`gallery.html`) zawiera własny lightbox – powiększanie obrazu w nakładce, zamykanej klawiszem `Esc`.

5.4 Asynchroniczność (HTMX)

Wysłanie formularza zapytania jest asynchroniczne dzięki **HTMX**. Formularz deklaruje atrybuty `hx-post`, `hx-target` i `hx-swap`, a widok rozpoznaje takie żądanie po nagłówku `HX-Request` i zwraca **fragment HTML** (a nie JSON): przy powodzeniu – sekcję podziękowania, przy błędach – ten sam formularz z komunikatami przy polach.

```
1 if form.is_valid():
2     inquiry = form.save()
3     _send_inquiry_confirmation(inquiry)
4     if request.headers.get("HX-Request"):
5         return render(request, "core/_inquiry_success.html", {"
            inquiry": inquiry})
6     messages.success(request, _("Twoje zapytanie zostało wysłane!
    "))
```

```

7     return redirect("inquiry-create")
8 else:
9     if request.headers.get("HX-Request"):
10        return render(request, "core/_inquiry_form.html", {"form"
            : form})

```

Listing 3: Rozgałęzienie odpowiedzi dla żądania HTMX (views/inquiries.py)

Podejście to (*progressive enhancement*) dokłada asynchroniczność do klasycznych szablonów serwerowych bez wprowadzania ciężkiego frameworka frontendowego.

6 Logika aplikacji

6.1 Formularze i walidacja

InquiryForm dziedziczy po `forms.ModelForm` – pola powstają z modelu Inquiry bez powielania definicji. Walidacja działa na kilku poziomach: typy pól (np. `EmailField`), reguły formularza (metody `clean_*`) oraz reguła modelu. Każdy formularz chroni token CSRF.

```

1 def clean_date_from(self):
2     date_from = self.cleaned_data.get("date_from")
3     if date_from and date_from < timezone.localtime():
4         raise forms.ValidationError(_("Data przyjazdu nie może by
            ć w przeszłości."))
5     return date_from
6
7 def clean(self):
8     cleaned = super().clean()
9     date_from, date_to = cleaned.get("date_from"), cleaned.get("
        date_to")
10    if date_from and date_to:
11        if date_to <= date_from:
12            self.add_error("date_to", _("Data wyjazdu musi być pó
                żniejsza..."))
13        elif (date_to - date_from).days > 30:
14            self.add_error("date_to", _("Pobyty nie może trwać dłu
                żej niż 30 dni."))
15    return cleaned

```

Listing 4: Walidacja wielopoziomowa (forms.py)

6.2 Dostęp do danych i optymalizacja ORM

Widoki pobierają dane wyłącznie przez ORM, z jawną optymalizacją liczby zapytań (przeciwdziałanie problemowi N+1):

```

1 # Lista pokoi: jedno dodatkowe zapytanie zamiast N (relacja 1:N)
2 Room.objects.filter(is_active=True).prefetch_related("images")
3
4 # Szczegóły pokoju: zdjęcia + udogodnienia (1:N oraz M:N)

```



```

5 Room.objects.prefetch_related("images", "amenities")
6
7 # Panel zapytań: JOIN z pokojem jednym zapytaniem (FK "w przód")
8 Inquiry.objects.select_related("room").order_by("-created_at")

```

Listing 5: Optymalizacja zapytań (views/rooms.py, views/inquiries.py)

6.3 Cache

Skonfigurowano backend `LocMemCache`. Buforowanie zastosowano w dwóch wariantach – niskopoziomowym (dane kontaktowe, które rzadko się zmieniają) oraz na poziomie całej strony (dekorator):

```

1 # 1. Cache niskopoziomowy - dane kontaktowe
2 prop = cache.get("property_info_contact")
3 if prop is None:
4     prop = PropertyInfo.objects.first()
5     cache.set("property_info_contact", prop, timeout=60 * 15) #
        15 min
6
7 # 2. Cache całej strony - dekorator
8 @cache_page(60 * 30) # strona /obiekt/ buforowana na 30 minut
9 def property_info(request): ...

```

Listing 6: Dwa wzorce cache (views/inquiries.py)

W zależnościach figuruje `django-redis` jako gotowa alternatywa (*drop-in*) do współdzielenia cache między procesami w środowisku produkcyjnym.

6.4 Poczta

Po zapisaniu zapytania system wysyła na adres gościa e-mail potwierdzający. Treść pochodzi z szablonu, a ewentualny błąd wysyłki jest przechwytywany i logowany – nie przerywa zapisu zapytania. W środowisku lokalnym SMTP kierowany jest do **MailHog** (kontener przechwytuje pocztę; podgląd pod `http://localhost:8025`).

```

1 def _send_inquiry_confirmation(inquiry):
2     body = render_to_string("core/emails/inquiry_confirmation.txt", {"inquiry": inquiry})
3     try:
4         send_mail(subject=..., message=body, from_email="noreply@u-alchemika.pl",
5                 recipient_list=[inquiry.email], fail_silently=False)
6         logger.info("E-mail wysłany na %s", inquiry.email)
7     except Exception as exc:
8         logger.error("Błąd wysyłki e-maila dla zapytania #%d: %s", inquiry.pk, exc)

```

Listing 7: Wysyłka potwierdzenia (views/inquiries.py)

6.5 Uwierzytelnianie i panel obsługi

Wykorzystano wbudowany system Django: `LoginView/LogoutView`, model `auth.User`, hashowanie haseł oraz walidatory ich siły. Panel zapytań chroni dekorator `@login_required` – próba wejścia bez sesji przekierowuje na `/login/` z parametrem `next`.

```

1 @login_required(login_url="login")
2 def inquiry_list(request):
3     inquiries = Inquiry.objects.select_related("room").order_by("
        -created_at")
4     return render(request, "core/inquiry_list.html", {"inquiries"
        : inquiries})

```

Listing 8: Ochrona panelu obsługi (`views/inquiries.py`)

6.6 Rejestrowanie zdarzeń (logger)

W `settings.LOGGING` skonfigurowano handler konsolowy z formatterem oraz logger apps na poziomie INFO. Rejestrowane są kluczowe zdarzenia: nowe zapytanie (`info`), nieprawidłowy formularz (`warning`), nieudana wysyłka poczty (`error`) oraz dostęp do panelu zapytań.

```

1 logger.info("Nowe zapytanie #%d od %s <%s> na pokój '%s' (%s-%s)"
2             ,
3             inquiry.pk, inquiry.full_name, inquiry.email,
4             inquiry.room, inquiry.date_from, inquiry.date_to)

```

Listing 9: Logowanie zdarzenia biznesowego (`views/inquiries.py`)

7 Internacjonalizacja (PL/DE)

Interfejs jest dwujęzyczny – polski (domyślny) i niemiecki – z przełącznikiem w pasku nawigacji. Konfiguracja obejmuje `LANGUAGES=[pl, de]`, `LocaleMiddleware` oraz `LOCALE_PATHS`. Przełączanie realizuje wbudowany widok `set_language`; wybór zapisywany jest w cookie `django_language`, bez prefiksów w adresie – ścieżki pozostają niezmienione.

```

<form action="{% url 'set_language' %}" method="post" class="lang
-switch">
    {% csrf_token %}
    <input name="next" type="hidden" value="{% request.path %}">
    <button name="language" value="pl">PL</button>
    <button name="language" value="de">DE</button>
</form>

```

Listing 10: Przełącznik języka w nawigacji (`base.html`)

Teksty interfejsu oznaczono `{% trans %}/{% blocktrans %}` w szablonach oraz `gettext/gettext_lazy` w kodzie Pythona (etykiety formularza, komunikaty, walidacje). Tłumaczenia niemieckie przechowuje katalog `backend/locale/de/LC_MESSAGES/django.po`, kompilowany do binarnego `.mo` przy starcie kontenera. Tłumaczeniem objęto *interfejs* aplikacji; treść redakcyjna z bazy (opisy pokoi, wpisy blogowe) pozostaje w języku polskim – jest to świadome ograniczenie zakresu, opisane w sekcji 10.

8 Konfiguracja i bezpieczeństwo

- **Konfiguracja przez zmienne środowiskowe** – dane wrażliwe (klucz, dostęp do bazy, DEBUG, ALLOWED_HOSTS) wczytuje `python-decouple`; w kodzie nie ma zaszytych sekretów.
- **Ochrona CSRF** – token w każdym formularzu, `CsrfViewMiddleware`.
- **Hasła** – hashowanie oraz walidatory siły (`AUTH_PASSWORD_VALIDATORS`).
- **Ochrona przed XSS** – automatyczne escapowanie zmiennych w szablonach.
- **Clickjacking** – `XFrameOptionsMiddleware` (nagłówek `X-Frame-Options`).

Zależności deklaruje `backend/requirements.txt` z kontrolą wersji (Django>=4.2,<5.0, `psycopg2-binary`, `python-decouple`, `django-redis`); instalacja odbywa się automatycznie podczas budowy obrazu Docker, co gwarantuje powtarzalne środowisko u każdego członka zespołu i na serwerze.

9 Konteneryzacja i uruchomienie

9.1 Środowisko (Docker Compose)

Całość jest skonteneryzowana. `docker-compose.yml` definiuje trzy serwisy:

```

1 services:
2   db:          # PostgreSQL 16 + healthcheck + wolumen
3     postgres_data
4     image: postgres:16
5     healthcheck:
6       test: ["CMD-SHELL", "pg_isready -U postgres -d u_alchemika"]
7
8   web:         # aplikacja Django
9     build: ./backend
10    ports: ["8000:8000"]
11    depends_on:
12      db:       { condition: service_healthy }
13      mailhog: { condition: service_started }
14    mailhog:    # przechwytywanie poczty (SMTP 1025, UI 8025)
15      image: mailhog/mailhog:latest
16      ports: ["1025:1025", "8025:8025"]
17
18 volumes:
19   postgres_data:

```

Listing 11: Serwisy aplikacji (fragment `docker-compose.yml`)

Serwis `web` startuje dopiero po przejściu healthchecku bazy. Przy starcie kontener wykonuje automatycznie migracje oraz kompilację tłumaczeń, po czym uruchamia serwer deweloperski. Komunikacja między kontenerami odbywa się po wewnętrznej sieci Dockera – `web` łączy się z bazą po nazwie `db` (wewnętrzny DNS), a na zewnątrz wystawione są jedynie zmapowane porty.

9.2 Uruchomienie lokalne

```
# 1. Klonowanie repozytorium
git clone https://github.com/Bulileusz/u-alchemika-system.git
cd u-alchemika-system

# 2. Uruchomienie kontenerow (migracje i kompilacja tłumaczen -
    automatycznie)
docker compose up --build

# 3. (Nowe okno) Zaladowanie danych testowych
docker compose exec web python manage.py loaddata seed

# 4. Utworzenie konta administratora (panel i /admin/)
docker compose exec web python manage.py createsuperuser
```

Listing 12: Pierwsze uruchomienie

Po uruchomieniu dostępne są: witryna publiczna (<http://localhost:8000/>), panel administracyjny ([/admin/](http://localhost:8000/admin/)) oraz skrzynka MailHog z podglądem maili (<http://localhost:8025/>).
Zatrzymanie: `docker compose down` (z flagą `-v` także reset bazy).

9.3 Wdrożenie zdalne

Konteneryzacja sprawia, że wdrożenie na serwer sprowadza się do uruchomienia tego samego `docker compose` na maszynie zdalnej. Względem konfiguracji deweloperskiej należy wprowadzić zmiany produkcyjne zestawione w tabeli 4.

Tabela 4: Zmiany konieczne przy wdrożeniu produkcyjnym

Element	Zmiana produkcyjna
DEBUG	ustawić na <code>False</code> .
DJANGO_SECRET_KEY	długi, losowy klucz podany przez zmienną środowiskową.
DJANGO_ALLOWED_HOSTS	domena/IP serwera.
Serwer WWW	zastąpić <code>runserver</code> serwerem produkcyjnym (np. <code>Gunicorn</code>) za reverse proxy (np. <code>Nginx</code>).
Pliki statyczne	<code>collectstatic</code> i serwowanie przez <code>Nginx</code> (lub <code>WhiteNoise</code>).
Poczta	realny serwer SMTP zamiast MailHog (dane z env).
Baza danych	trwały wolumen lub usługa zarządzana; silne hasło.
HTTPS	certifikat TLS (np. <code>Let's Encrypt</code>) na reverse proxy.

Uwaga: bieżąca wersja repozytorium jest skonfigurowana pod środowisko deweloperskie (serwer `runserver`, MailHog, domyślne sekrety). Powyższe kroki są konieczne do bezpiecznego wdrożenia produkcyjnego.

10 Podsumowanie i decyzje projektowe

1. **Framework porządkuje pracę.** Django pozwoliło skupić się na logice obiektu noclegowego zamiast na „hydraulicie” (routing, ORM, formularze, panel admina, bezpieczeństwo). Mechanizmy gotowe z pudełka (CSRF, hashowanie haseł, escapowanie XSS) podniosły bezpieczeństwo bez dodatkowego nakładu.
2. **Konteneryzacja eliminuje problem „u mnie działa”.** Docker Compose zapewnił identyczne, powtarzalne środowisko dla całego zespołu i uprościł ścieżkę do wdrożenia – ten sam plik kompozycji działa lokalnie i na serwerze.
3. **Technologia proporcjonalna do skali.** Zamiast ciężkiego frameworka frontendowego wybrano HTMX, który dokłada asynchroniczność do szablonów serwerowych minimalnym kosztem – dla pojedynczego interaktywnego formularza było to rozwiązanie adekwatne.
4. **Świadome ograniczenie zakresu.** Rezygnacja z pełnej rezerwacji online oraz z tłumaczenia treści redakcyjnej pozwoliła dostarczyć kompletny, dopracowany zakres zamiast wielu niedokończonych funkcji.
5. **Optymalizacja ma znaczenie już na małej skali.** Zastosowanie `prefetch_related/select_related` oraz cache dla danych rzadko zmienianych pokazało, jak prostymi środkami ogranicza się liczbę zapytań do bazy.
6. **Kierunki rozbudowy.** Naturalne rozszerzenia to: wdrożenie produkcyjne za reverse proxy z HTTPS, integracja z zewnętrznym API (np. mapa okolicy lub pogoda na stronie atrakcji) oraz udostępnienie własnego API z listą pokoi, pełne tłumaczenie treści (`django-modeltranslation`) i podpięcie współdzielonego cache Redis (zależność jest już dołączona).

Metryka	Wartość
Aplikacje Django	2 (core, content)
Strony publiczne	10
Encje modelu danych	9
Serwisy kontenerów	3 (web, db, mailhog)
Obsługiwane języki	2 (PL, DE)

A Skorowidz zagadnień warstwy internetowej

Pomocniczy skorowidz wiążący zagadnienia omawiane na kursie z miejscem ich realizacji w systemie oraz sekcją niniejszego dokumentu. Służy szybkiej nawigacji – nie jest listą kontrolną.

Zagadnienie	Realizacja w systemie	Sekcja
Framework MVC/MVT	Django 4.2	3
Framework CSS	Bootstrap 5.3 (CDN)	5
HTML / dziedziczenie	<code>base.html</code> + <code>partiale</code>	5
CSS własny	<code>static/css/style.css</code>	5
JavaScript	<code>inquiry.js</code> , <code>lightbox</code> galerii	5
RWD	<code>viewport</code> + siatka + <code>media queries</code>	5
Asynchroniczność	HTMX (fragmenty HTML)	5
Routing / pretty URLs	<code>urls.py</code> , <code><slug></code>	3
ORM	<code>modele</code> + <code>prefetch/select_related</code>	6.2
Baza danych	PostgreSQL 16	4
Formularze	<code>InquiryForm</code> + walidacja	6
Cache	<code>LocMemCache</code> , <code>@cache_page</code>	6
Poczta	<code>send_mail</code> → MailHog	6
Uwierzytelnianie	<code>LoginView</code> , <code>@login_required</code>	6
Logger	<code>LOGGING</code> + <code>logger.*</code>	6
Lokalizacja	przełącznik PL/DE, <code>gettext</code>	7
Dependency manager	<code>pip</code> + <code>requirements.txt</code>	8
Deployment	Docker Compose	9